Rendu 1 : Dossier de Lancement — GreenOps

1. Informations Générales & Identification de l'Équipe

1.1 Présentation du Projet et Dépôt Source

- **Intitulé du projet :** GreenOps (Plateforme SaaS d'évaluation et de pilotage de l'empreinte environnementale logicielle).
- **Dépôt principal :** https://github.com/Belvis/GreenOps
- **Date de livraison :** 10 novembre 2026



1.2 Identification des Membres et Comptes Développeurs

Le projet est réalisé par un groupe de 4 étudiants de Master. L'ingénierie collaborative repose sur une attribution stricte d'une fonctionnalité majeure de bout en bout (Front-end, Back-end API REST et Tests unitaires JUnit) par personne.

Nom & Prénom	Numéro Étudiant	Compte GitHub	Rôle & Feature Majeure Dévolue
AGBOTON Belvis	44001686	github.com/Belvis	**Feature Majeure 1 :** Analyseur d'impact du code & CI/CD sur dépôts publics (Module 2)
RAMALINGAME Shiamamadanki	46018608	github.com/Shiama-ramalingame	**Feature Majeure 2 :** Surveillance planifiée (Cron), alertes emails (Module 4) & Socle transversal / Admin (Module 0)
HAFSAT Marwa	45007615	github.com/Marwa-Hafsati	**Feature Majeure 3 :** Audit Web d'éco-conception en temps réel & Recommandations officielles (Module 1)
PINEAU Faizi	43014859	github.com/GitFaiziT6	**Feature Majeure 4 :** Simulateur Cloud éco-responsable, critères RGPD & Optimisation (Module 3)

1.3 Vision du Projet et Caractère Innovant

L'expansion rapide des infrastructures logicielles et l'automatisation industrielle des déploiements génèrent une empreinte écologique numérique croissante et largement méconnue. La plupart des solutions existantes restent cloisonnées : elles mesurent soit la vitesse de rendu côté navigateur, soit la facture financière cloud, sans corrélation globale.

GreenOps comble cette rupture en proposant une plateforme SaaS unifiée d'évaluation environnementale logicielle couvrant l'ensemble du cycle de vie applicatif :

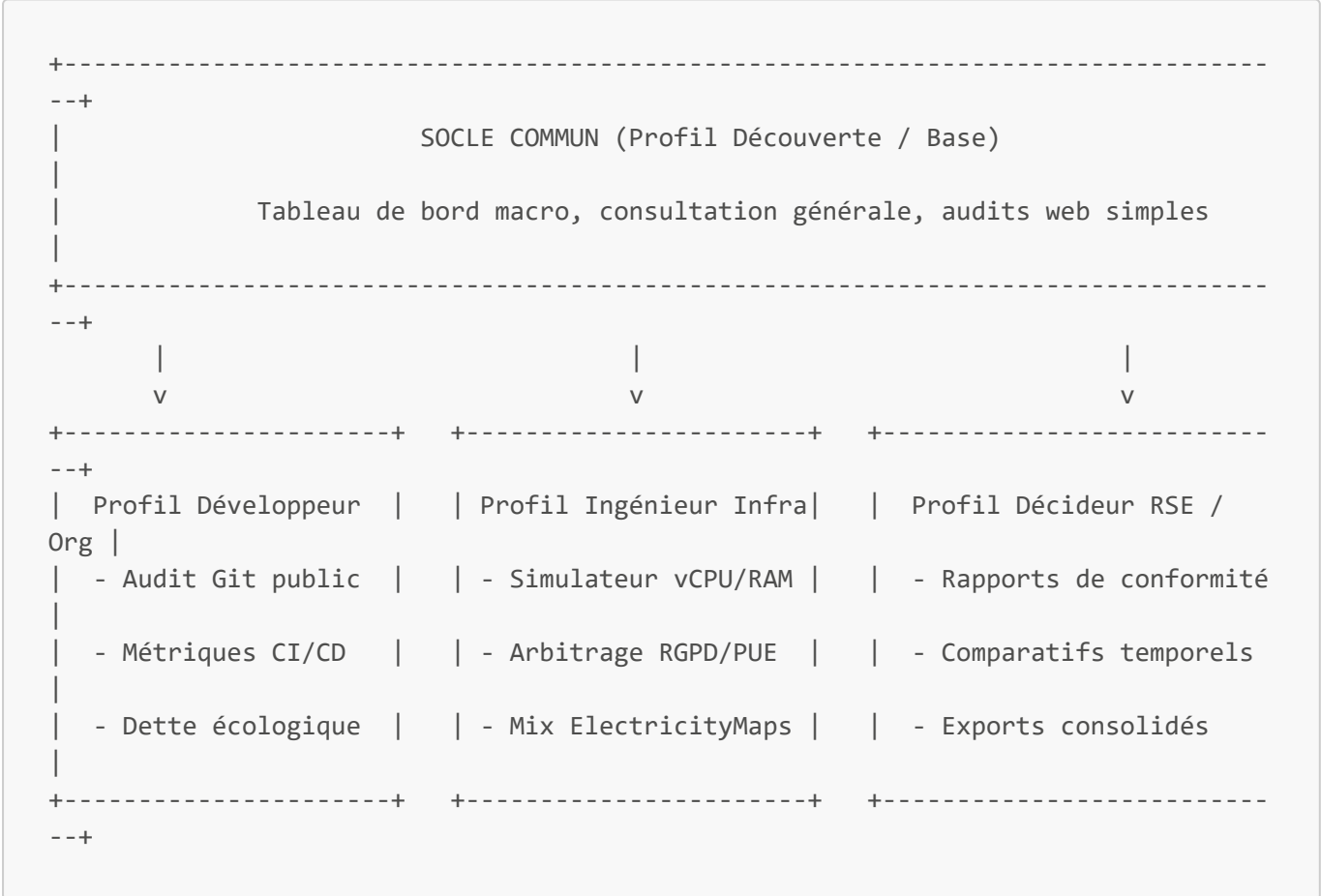
1. **Approche Holistique de Bout en Bout :** GreenOps rassemble sur une même interface l'audit du code déployé (poids du DOM, requêtes, mix énergétique de l'hébergeur), l'impact amont des usines de développement (coût machine des pipelines CI/CD sur runners) et l'arbitrage en amont des architectures serveurs.
2. **Double Arbitrage Inédit Écologie & Souveraineté (RGPD) :** Le simulateur d'infrastructure innove en refusant le compromis naïf du "100 % bas carbone". Il intègre une matrice de conformité réglementaire stricte pour interdire ou pénaliser les transferts de données hors Union Européenne (soumis à des législations extraterritoriales de type Cloud Act), identifiant ainsi le point d'équilibre optimal entre mix énergétique décarboné et respect absolu de la vie privée.
3. **Restitution Factuelle et Certifiée :** Contrairement aux calculateurs arbitraires, GreenOps s'adosse exclusivement aux diagnostics standardisés officiels (Google Lighthouse / PageSpeed, The Green Web Foundation, Electricity Maps) sans extrapolation fantaisiste des opportunités de refactorisation.

\newpage

2. Scénarios d'Usage & Répartition des Responsabilités

2.1 Scénarios d'Usage (Personas & Cas d'Utilisation Métier)

La plateforme GreenOps adopte une architecture SaaS multi-rôles où les privilèges sont gérés de manière cumulative par contrôle d'accès basé sur les rôles (RBAC), attribués centralement par l'administrateur.



- **Scénario 1 : L'Ingénieur DevOps / Développeur (Profil Opérationnel)**

    - *Contexte :* Le développeur souhaite assainir son usine logicielle et mesurer le coût carbone induit par ses tests automatisés.
    - *Parcours :* Il saisit l'URL HTTPS publique de son dépôt GitHub/GitLab. GreenOps analyse l'historique des workflows CI/CD, calcule l'énergie consommée par les runners en kWh et gCO2eq, identifie les dépendances lourdes et fournit des axes d'optimisation actionnables (ex. factorisation d'étapes de build, mise en cache Maven).
- **Scénario 2 : L'Architecte Système / Ingénieur Cloud (Profil Infrastructure)**
    - *Contexte :* L'architecte dimensionne une grappe d'instances et doit choisir une région d'implantation respectant les engagements RSE et légaux de l'organisation.
    - *Parcours :* Il configure les spécifications techniques (vCPU, RAM, stockage, profil temporel). Le système interroge le mix énergétique local en direct, évalue la conformité RGPD de la zone et préconise l'alternative optimale garantissant la plus faible émission sous juridiction protectrice.
- **Scénario 3 : Le Responsable RSE / Décideur Numérique (Profil Stratégique)**
    - *Contexte :* Le responsable doit produire un bilan extra-financier de l'impact des services web de l'entreprise.
    - *Parcours :* Il accède aux historiques d'audits web, compare les versions avant et après refonte technique et télécharge un rapport PDF certifié synthétisant le score EcoIndex, la consommation d'eau et les gains environnementaux.
- **Scénario 4 : L'Administrateur SaaS (Super-Admin)**
    - *Contexte :* Supervision opérationnelle et gouvernance de la plateforme.
    - *Parcours :* Depuis une console dédiée, il valide les nouveaux comptes, assigne les rôles avancés aux collaborateurs, supervise les quotas restants des API externes et ajuste les coefficients d'émission carbone en base de données.

2.2 Répartition Fonctionnelle des Développements

Chaque étudiant prend en charge l'implémentation complète (Front, Back et Tests) d'une tranche verticale :

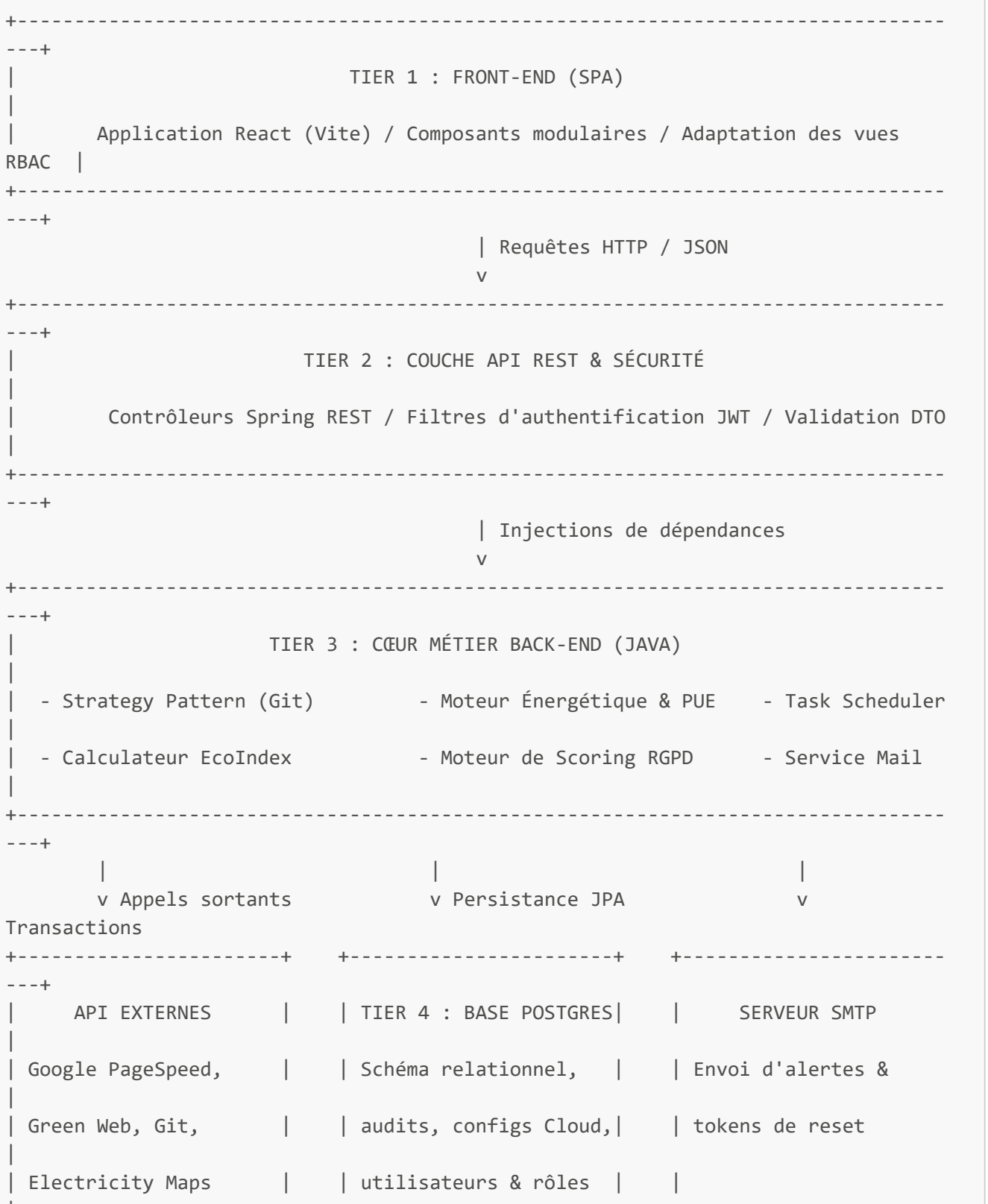
- **Feature Majeure 1 — Analyseur d'impact Code & CI/CD (AGBOTON Belvis) :** Ingestion et traitement des flux publics GitHub/GitLab via le patron de conception *Strategy*, conversion des durées de runners en énergie consommée, détection des dettes de build et affichage des graphiques d'évolution sous React. Tests unitaires des parseurs et des séries temporelles.
- **Feature Majeure 2 — Surveillance planifiée & Notifications (RAMALINGAME Shiamamadanki) :** Ordonnanceur asynchrone Cron en tâche de fond, service d'alerte transactionnelle 100 % email via SMTP, moteur d'analyse des deltas de performance, couplé au socle d'authentification JWT (gestion du profil et réinitialisation de mot de passe par token à usage unique) et à la console de gestion administrative des rôles SaaS.
- **Feature Majeure 3 — Audit Web d'éco-conception en temps réel (HAFSAT Marwa) :** Intégration des API Google PageSpeed et Green Web Foundation, extraction du DOM via Jsoup, calcul d'EcoIndex, affichage des opportunités officielles Lighthouse et module de comparaison temporelle avant/après.
- **Feature Majeure 4 — Simulateur Cloud & Matrice RGPD (PINEAU Faizi) :** Modélisation énergétique thermique (PUE, vCPU, RAM), connecteur d'intensité carbone en temps réel (Electricity Maps), développement de l'algorithme multicritère croisant émissions carbone et conformité RGPD, et sélection automatisée de la recommandation optimale.

\newpage

3. Architecture Logicielle & Frameworks Envisagés

3.1 Architecture Système à 4 Niveaux (Front - API - Back - BD)

L'application GreenOps adopte une architecture distribuée, modulaire et hautement découplée permettant le travail collaboratif sans interblocage technique.



```
+----------------------+    +----------------------+    +----------------------
---+
```

3.2 Frameworks et Choix Technologiques Envisagés

Afin d'assurer la maintenabilité, l'extensibilité et le respect des critères académiques, les choix technologiques suivants ont été retenus pour la phase d'implémentation :

- **Back-end : Java 21 & Framework Spring Boot 3.x**
  - *Spring Web :* Exposition d'une API REST robuste avec sérialisation/désérialisation JSON stricte.
  - *Spring Security & jjwt :* Gestion fine du contrôle d'accès basé sur les rôles (RBAC) via jetons JWT sans état (*stateless*).
  - *Spring Data JPA / Hibernate :* Modélisation et abstraction de la couche d'accès aux données.
- **Gestionnaire de dépendances et Build : Maven**
  - Automatisation du cycle de vie logiciel, gestion des dépendances externes et intégration continue des plugins de vérification de code.
- **Front-end : React (avec Vite)**
  - Développement d'une interface monopage (SPA) fluide, modulaire et dynamique. L'état applicatif gère dynamiquement l'affichage conditionnel des composants selon le rôle de l'utilisateur connecté.
- **Base de données : PostgreSQL**
  - Base relationnelle performante, idéale pour gérer l'intégrité référentielle des utilisateurs, l'historique volumineux des audits temporels et les référentiels de conformité légale.
- **Validation & Tests : JUnit 5 & Mockito**
  - Rédaction systématique de tests unitaires pour chaque composant métier critique (calculateurs carbone, parseurs d'arbres syntaxiques, algorithme de scoring RGPD).
  - Isolation stricte des tests d'intégration par le bouchonnage (*mocking*) des API externes via Mockito pour garantir l'indépendance de la suite de tests face aux fluctuations réseau.

3.3 Connecteurs et Protocoles Externes

- **Google PageSpeed Insights API :** Récupération des données brutes de performance web et extraction des diagnostics d'éco-conception officiels Lighthouse.
- **The Green Web Foundation API :** Vérification de l'alimentation des datacenters en énergies renouvelables.
- **Endpoints publics GitHub & GitLab :** Consultation sans authentification des durées de jobs CI/CD et métadonnées de versioning.
- **Electricity Maps API :** Ingestion en temps réel des facteurs d'intensité carbone régionaux ($\mathrm{gCO_2eq/kWh}$).
- **Service SMTP (JavaMail) :** Routage transactionnel des emails de réinitialisation et des alertes de dérive métrique.

\newpage

4. Protocole d'Usage de l'IA & Stratégie DevOps

4.1 Protocole d'Usage de l'Intelligence Artificielle

Dans le cadre de ce projet de Master, l'utilisation des assistants d'Intelligence Artificielle générative est strictement encadrée par une charte d'équipe visant à garantir l'authenticité académique, la compréhension technique intégrale du code et la sécurité logicielle.

- **Cas d'usage autorisés :**
  - Assistance à la formalisation et structuration documentaire (mise en page Markdown, correction syntaxique).
  - Génération d'échantillons de données de test et de structures JSON fictives pour les tests unitaires.
  - Brainstorming d'architectures logicielles et exploration des patrons de conception appropriés.
  - Aide à la détection de bogues sur des messages d'erreur ou traces d'exécution complexes.
- **Interdictions formelles et engagements éthiques :**
  - *Interdiction de génération aveugle :* Tout composant inséré dans le dépôt doit être intégralement compris, revu et explicable à l'oral par son auteur désigné lors des soutenances.
  - *Sécurité et confidentialité :* Interdiction absolue de soumettre aux modèles d'IA des identifiants sensibles, mots de passe de test ou clés d'API.
  - *Traçabilité :* Toute portion de code ou d'algorithme ayant bénéficié d'une assistance IA significative doit faire l'objet d'une mention explicite dans les commentaires de validation (*commits*) ou la documentation technique associée.

4.2 Stratégie de Versioning sous Git & Organisation du Dépôt

Le développement collaboratif à 4 impose une rigueur d'ingénierie logicielle pour éliminer les régressions et structurer les livraisons.

- **Structure physique du dépôt (`/Belvis/GreenOps`) :**

```
/
├── doc/                    # Documentation versionnée (dossier_lancement.md, PDF)
├── backend/                # Code source Java / Spring Boot, pom.xml, tests JUnit
│   ├── src/main/java/      # Architecture en couches (controllers, services, entities)
│   └── src/test/java/      # Suites de tests unitaires et mocks Mockito
├── frontend/               # Application cliente React (Vite), composants, styles
├── .gitignore              # Exclusion des fichiers compilés (target/, node_modules/)
├── LICENSE                 # Licence logicielle MIT
└── README.md               # Présentation, badges de contrôle, guide d'installation
```

- **Stratégie de Branches (*Git Feature Branching*) :**
  - `main` : Branche de production protégée, hébergeant uniquement les versions stables livrables.
  - `develop` : Branche centrale d'intégration continue des fonctionnalités validées.

    - `feature/<nom-etudiant>-<nom-module>` : Branches de travail isolées par fonctionnalité (ex. `feature/belvis-ci-analyzer`, `feature/faizi-cloud-simulator`).
- **Politique d'Intégration et de Validation :**
    - Interdiction stricte de pousser directement sur `main` ou `develop`.
    - Toute fusion fait l'objet d'une *Pull Request* (PR) documentée.
    - Règle de revue croisée : Chaque PR nécessite la revue et l'approbation formelle d'au moins un autre membre de l'équipe et le succès complet de la suite de tests unitaires automatisés avant intégration.

4.3 README, Badges & Calendrier Prévisionnel des Livraisons

Le fichier `README.md` à la racine servira de vitrine technique en intégrant dynamiquement les badges d'état (CI/CD GitHub Actions, couverture de code JaCoCo, rapport qualité SonarCloud).

- **Jalon 1 (Semaines 1 à 3) :** Mise en place du socle technique, configuration du modèle de données PostgreSQL, implémentation de l'authentification JWT et squelettes des contrôleurs REST.
- **Jalon 2 (Semaines 4 à 6) :** Développement des moteurs métier par fonctionnalité majeure (PageSpeed, Strategy Git, simulateur Cloud) et couverture JUnit associée (> 80 %).
- **Jalon 3 (Semaines 7 à 8) :** Intégration de l'interface React, implémentation du scheduler d'alertes emails, tests d'intégration globaux et préparation de la soutenance finale.